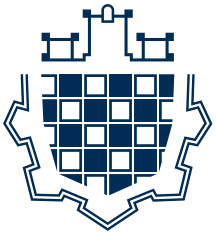

UNIVERSITY OF MANNHEIM

School of Business Informatics and Mathematics

Test Suite Minimisation in LASSO Using Stimulus-Response Matrices

Bachelor Thesis

submitted: 01.December 2025

by: Laith Philipp Sandouk
laith.sandouk@students.uni-mannheim.de
born May 15th, 2005
in Mannheim

Student ID Number: 1993811

Chair of Software Engineering
School of Business Informatics and Mathematics
University of Mannheim
D – 68159 Mannheim
Phone: +49 621-181-3912, Fax +49 621-181-3909
Internet: http://swt.informatik.uni-mannheim.de

Abstract

Large-scale industrial software systems face significant challenges in executing test suites efficiently. In a commercial SAP environment, Bach et al. report test suites comprising more than 130,000 tests requiring hundreds of hours of sequential runtime [3]. Such execution times render continuous testing computationally infeasible and economically prohibitive. The study further revealed extensive redundancy within these test suites, implying that substantial execution cost yields only marginal additional coverage benefit. However, this is not an isolated case, test-suite redundancy is a widespread and well-documented issue in industrial software development. While this computational burden cannot be eliminated entirely, its impact can be substantially reduced through test-suite minimisation, which systematically removes redundant tests with respect to defined adequacy criteria such as statement coverage, branch coverage, or mutation score [23]. Test-suite minimisation is particularly critical for regression testing, where previously passing tests are re-executed after code changes to detect newly introduced defects. However, naive reduction approaches risk compromising fault-detection effectiveness, as tests with unique defect-revealing capabilities may be inadvertently excluded.

This thesis investigates test-suite minimisation in the context of the *Large-Scale Software Observatorium* (LASSO) [12], a research platform for automated selection, execution, and comparative analysis of software systems. LASSO integrates open-source repositories and combines static and dynamic analyses within a unified framework designed for language independence. Its current implementation focuses on Java and Python systems and enables empirically grounded studies of *Big Code* ecosystems (large, diverse corpora of source code enriched with metadata such as bug-fix histories, version control traces, and execution outcomes [2]).

Our approach leverages *Stimulus–Response Matrices* (SRMs) [12], a two-dimensional representation mapping test stimuli to observed execution outcomes. We enrich SRM entries with per-test coverage information and utilise the SRM's existing

mutation data, enabling minimisation algorithms to operate on fine-grained behavioural information rather than language-specific code structures.

Through a targeted literature survey encompassing greedy heuristics, exact optimisation, search-based, and data-driven minimisation strategies, we comparatively assessed candidate algorithms along six established criteria: SRM compatibility, coverage retention, reduction effectiveness, mutation-coverage preservation, computational scalability, and implementation feasibility. The Mutation-Aware Enhanced HGS (MA-EHGS) algorithm emerged as the most promising candidate. MA-EHGS extends the empirically validated Enhanced HGS baseline [8] by integrating mutation weighting to balance coverage preservation and fault-detection power [10].

We successfully integrated MA-EHGS into LASSO and validated the implementation on real-world SRMs. Our evaluation achieved reductions within the ranges reported in prior studies, while maintaining high coverage and mutation-scores [8, 10]. The algorithm executes in $O(nm)$ time, ensuring computational feasibility for large-scale software analysis workflows.

Contents

List of Figures

List of Tables

1. Introduction

Software testing ensures that program modifications do not introduce defects. As systems evolve, test suites grow and increase execution cost and maintenance effort. Modern suites often combine human-written, automatically generated, and AI-generated tests; tools such as Randoop [18] and EvoSuite [7] can produce thousands of tests, and large industrial suites may require days or weeks to execute [20]. Although such suites achieve high coverage and mutation scores, they frequently contain redundant tests whose requirements are subsumed by others, resulting in prolonged execution, increased costs, and elevated maintenance overhead.

Test-suite minimisation addresses this issue by systematically removing redundant tests while preserving fault-detection capability [23]. Techniques range from *coverage-preserving* approaches, which maintain all coverage requirements, to *coverage-relaxing* approaches, which accept limited coverage loss for stronger reduction. While minimisation can significantly reduce computational demand and feedback latency, overly aggressive reduction risks discarding tests with unique defect-revealing behaviour. Effective minimisation therefore requires balancing reduction efficiency with fault-detection retention.

1.1. Background and Motivation

Testing remains a major cost driver in software engineering; classical studies estimate that verification and validation account for up to half of total development effort [4]. The cost pressure persists in modern continuous-integration environments, where automated or AI-generated suites may contain tens of thousands of tests [20, 3]. Executing all tests after each change is often infeasible, motivating techniques that reduce suite size while retaining quality. Test suite minimisation has been exploring this issue for decades, with numerous algorithms proposed to

identify and eliminate redundant tests [23], by leveraging coverage and quite recently mutation information as a proxy for fault-detection capability [10]. Details on coverage and mutation testing are provided in Chapter 2.

1.2. LASSO Context

The *Large-Scale Software Observatorium* (LASSO) [12] is a research platform for scalable analysis of large software corpora (*Big Code*). LASSO automates the extraction of static properties and dynamic execution behaviour from open-source systems and enables reproducible pipelines through the *LASSO Scripting Language* (LSL). This makes it a central infrastructure for empirical software engineering and for evaluating emerging techniques such as AI-assisted code generation.

LASSO represents system behaviour through *Stimulus–Response Matrices* (SRMs) [12], where rows denote test stimuli, columns denote systems, and cells contain structured response objects. SRMs allow language-independent extraction of coverage metrics, mutation scores, and behavioural characteristics, enabling systematic comparison of functionally equivalent implementations. Section 2.2 provides formal definitions.

Despite supporting large-scale behavioural and coverage analysis, LASSO currently lacks integrated test-suite minimisation. Redundant test executions accumulate significant computational overhead across the platform. Integrating minimisation would reduce this overhead while preserving the behavioural coverage required for reliable system analysis.

1.3. Research Objectives and Questions

This thesis investigates how test-suite minimisation can be integrated into LASSO using its SRM abstraction. The study is guided by:

1. **RQ1:** Which minimisation techniques demonstrate the highest practical effectiveness in the literature?
2. **RQ2:** Which evaluation criteria best suit minimisation within LASSO's SRM-based environment?

3. **RQ3:** Which approach best balances reduction effectiveness, coverage preservation, mutation preservation and scalability?
4. **RQ4:** How can the selected approach be integrated into LASSO while maintaining language independence?

1.4. Implementation Strategy

This thesis performs minimisation directly on SRMs rather than relying on external tools, which typically require source-code access or language-specific instrumentation. SRMs already encapsulate all data required for behavioural minimisation—test identifiers, execution results, coverage information, and mutation data—allowing language-independent and scalable processing while ensuring seamless integration into LASSO's analysis pipeline.

1.5. Thesis Structure

The remainder of this thesis is organised as follows:

Chapter 2 introduces coverage, mutation testing, SRMs, and formally defines the minimisation problem.

Chapter 3 surveys greedy, exact, search-based, and data-driven minimisation techniques.

Chapter 4 defines evaluation criteria (C1–C6) and the scoring framework.

Chapter 5 applies the framework to select the Mutation-Aware Enhanced HGS approach.

Chapter 6 describes the implementation and integration into LASSO.

Chapter 7 discusses limitations, trade-offs, and deployment considerations.

Chapter 8 concludes and outlines future work.

2. Fundamentals

This chapter establishes the theoretical foundation for this work. It introduces the core concepts of test coverage and mutation testing, explains the *Stimulus–Response Matrix* (SRM) representation used within LASSO, and formalises the test-suite minimisation problem, including trade-offs between coverage preservation and reduction strength.

2.1. Test Coverage and Mutation Score

Test coverage quantifies the proportion of program elements exercised by a test suite [24]. Coverage metrics indicate structural thoroughness but provide limited insight into fault-detection capability.

2.1.1. Coverage Metrics

Coverage criteria define program "parts" at different granularity levels [24]. Common coverage types include:

- **Instruction Coverage:** proportion of bytecode instructions executed.
- **Line Coverage:** proportion of source lines executed.
- **Branch Coverage:** proportion of conditional branches whose true/false outcomes are both executed.
- **Method Coverage:** proportion of methods invoked at least once.
- **Complexity Coverage:** proportion of independent execution paths exercised (cyclomatic complexity).
- **Class Coverage:** proportion of classes instantiated or whose static methods execute.

Multi-level coverage captures execution at different granularities. High line coverage with low branch coverage may miss conditional faults, while high method coverage without path coverage may miss defects in complex methods. Combining several metrics provides a more complete assessment of test quality.

2.1.2. Mutation Score

Mutation testing measures test effectiveness by introducing small syntactic changes (*mutants*) into the program and observing whether tests detect the injected faults [11]. The *mutation score* is defined as:

$$\text{Mutation Score} = \frac{\text{Number of mutants killed}}{\text{Total number of mutants}} \times 100\,\%.$$

A mutant is *killed* if the test causes observable behavioural divergence from the original program.

Mutation Types. Two principal mutation styles are distinguished [17]:

1. **Weak Mutation:** detects state deviations immediately after a mutated statement executes.
2. **Strong Mutation:** detects externally visible behavioural differences at program output.

Weak mutation captures local sensitivity to code changes; strong mutation quantifies global behavioural divergence. Mutation scores thereby approximate a suite's fault-detection capability.

2.2. Stimulus–Response Matrices

Stimulus–Response Matrices (SRMs) constitute LASSO's central data structure for *Test-Driven Software Experimentation* [12]. They provide a unified, language-independent format for recording and comparing run-time behaviour under controlled stimuli.

2.2.1. Sequence and Actuation Sheets

Sequence Sheets describe ordered stimuli applied to systems under test. Each row represents one method invocation with inputs and expected outputs. **Actuation Sheets** record the corresponding observed outputs and other run-time data during execution.

Stimulus (Sequence Sheet)

Out	Operation	Service	Input
create	`create`	`'Stack`	
–	`push`	A1	42
42	`pop`	A1	

Observed Behaviour (Actuation Sheet)

Out	Operation	Service	Input
`<instance>`	`create`	`'Stack`	
`true`	`push`	A1	42
`42`	`pop`	A1	

Figure 2.1.: Relationship between a sequence sheet (stimulus) and its corresponding actuation sheet (observed behaviour).

2.2.2. From Sequence Sheets to SRMs

LASSO generalises stimulus-to-actuation mappings into multidimensional *Stimulus–Response Matrices* (SRMs):

$$M = [M_{ij}], \qquad M_{ij} = \text{Actuation}(t_i, v_j),$$

where each row t_i represents a stimulus (test), each column v_j a system variant, and each cell M_{ij} stores execution outcomes and associated analysis data such as coverage and mutation attributes.

	System 1	System 2	System 3	System 4
Test 1	pass	fail	pass	fail
Test 2	pass	pass	pass	pass
Test 3	pass	fail	fail	pass

Figure 2.2.: Example SRM fragment showing per-test outcomes across different system variants. Behavioural discrepancies can be used to compute mutation scores and coverage information.

SRMs unify functional results, coverage data, and mutation outcomes in a single matrix, enabling language-independent behavioural analysis.

2.3. The Test-Suite Minimisation Problem

2.3.1. Concept and Objectives

Test-suite minimisation seeks to remove redundant tests from an existing suite while maintaining sufficient coverage and fault-detection capability [23]. Given a suite $T = \{t_1, \ldots, t_n\}$ and a set of coverage requirements $R = \{r_1, \ldots, r_m\}$, each test t_i covers a subset $C(t_i) \subseteq R$. The objective is to find a smaller subset $S^* \subseteq T$ that provides comparable or identical coverage.

Two major formulations exist:

- **Coverage-preserving minimisation:** guarantees full coverage, $C(S^*) = C(T)$. It ensures no requirement is lost but often limits reduction potential.
- **Coverage-relaxing minimisation:** allows limited loss of coverage, $C(S^*) \subset C(T)$, to achieve greater reduction. This variant trades fault-detection certainty for smaller suite size.

Both forms address the same optimisation tension: *minimise suite size while retaining sufficient adequacy.* The general optimisation can be expressed as

$$S^* = \arg\min_{S \subseteq T} |S| \quad \text{subject to} \quad |C(S)| \geq \gamma \cdot |C(T)|,$$

where $0 < \gamma \leq 1$ controls the acceptable coverage retention threshold ($\gamma = 1$ for coverage-preserving, $\gamma < 1$ for coverage-relaxing).

2.3.2. Computational Complexity and Trade-offs

The coverage-preserving variant corresponds to the *minimum set-cover problem*, which is NP-complete [23]. No polynomial-time algorithm guarantees optimality for arbitrary inputs, and practical implementations rely on heuristics or approximations [9, 5]. Some greedy heuristics achieve linear complexity $O(nm)$ in practice, balancing scalability and quality.

Coverage-relaxing algorithms may yield stronger reductions but risk discarding unique defect-revealing tests, potentially lowering mutation scores or missing rare execution paths. Hence, every minimisation approach must balance the trade-off between computational efficiency, reduction rate, and fault-detection retention.

Original Test Suite

$T = \{t_1, \ldots, t_n\}$

$|T| = n$ tests

$C(T) = \bigcup_{i=1}^{n} C(t_i)$

Complete coverage

Reduced Test Suite

$S^* \subseteq T$

$|S^*| < n$ tests

$C(S^*) = C(T)$ (preserving)

or $C(S^*) \subset C(T)$ (relaxing)

Optimisation Goal

Minimise: $|S^*|$

Subject to: $C(S^*) \geq \gamma \cdot C(T)$, where $0 < \gamma \leq 1$

Complexity: NP-complete (set-cover equivalent)

Figure 2.3.: The test-suite minimisation problem formulated as an optimisation balancing suite size and coverage retention.

2.3.3. Minimisation within LASSO

In LASSO, each test corresponds to one SRM row. Coverage and mutation data stored in SRM cells provide the requirement mappings $C(t_i)$. Minimisation thus operates directly on SRMs by selecting a subset of rows S^* that preserves coverage and mutation adequacy across all variants:

$$C(S^*)_{\text{SRM}} \geq \gamma \cdot C(T)_{\text{SRM}}.$$

This SRM-based formulation supports both coverage-preserving and coverage-relaxing strategies while remaining language-independent, enabling scalable experimentation across diverse software systems.

2.3.4. Algorithmic Categories

Following established classifications [23, 19, 3], algorithms group into four categories: **greedy heuristics** (classical greedy, HGS [9]), **exact optimisation** (ILP, constraint-satisfaction [14]), **search-based methods** (genetic algorithms [23]), **data-driven methods** (clustering, overlap-aware [3]). Detailed explanation in Chapter 3.

3. Minimisation Algorithms

This chapter surveys test-suite minimisation algorithms across four categories: greedy heuristics, exact optimisation, search-based approaches, and data-driven techniques.

3.1. Greedy Heuristics

Greedy heuristics iteratively select tests covering the most uncovered requirements until complete coverage is achieved [23, 19], operating in polynomial time with predictable performance.

3.1.1. Classical Greedy

Classical greedy iteratively selects tests covering the most uncovered requirements [5, 23], achieving $O(nm)$ complexity with $O(\ln m)$ approximation ratio. Empirical studies on JavaScript applications show 70% reduction (median) maintaining 100% structural coverage with zero to minimal fault-detection loss for adequate reduction [10]. When allowing inadequate reduction, mutation scores may drop up to 21% [10].

3.1.2. Harrold–Gupta–Soffa

HGS prioritises requirements by rarity [9], operating in phases by cardinality with runtime complexity of $O(|T| \cdot \max(|T_i|))$ [23], where $|T|$ is the size of the original test suite and $\max(|T_i|)$ represents the cardinality of the largest group of test cases. HGS produces suites comparable in size to classical greedy with statistically similar fault-detection capability ($p \gg 0.05$) [23].

3.1.3. Delayed Greedy

Delayed greedy [21] uses concept lattices to eliminate redundancies before greedy selection. It produces suites equal to or smaller than classical greedy in empirical evaluation [21, 23], achieving approximately 65% reduction [10] while maintaining fault-detection capability. Polynomial complexity, though exact bound unstated [21].

3.1.4. Enhanced HGS

Enhanced HGS [8] operates on a requirements matrix in two phases: (1) select tests uniquely covering any requirement, (2) iteratively select tests maximising uncovered requirements. Time complexity is $O(nm)$, matching classical greedy. Empirical validation demonstrated that Enhanced HGS produces smaller suites (73.66% average reduction) than both HGS (69.01%) and classical greedy algorithms while maintaining coverage [8].

3.1.5. Greedy Heuristics with Overlap-Awareness

While classical greedy, HGS, and delayed greedy prioritise test coverage, overlap-aware variants enhance these methods by penalising tests that duplicate existing coverage [3]. This approach is particularly effective in time-budget constrained environments (test case prioritisation scenarios) rather than pure minimisation. Bach et al. demonstrated that overlap-aware methods achieve 50% higher code coverage within fixed time budgets versus classical greedy, completing under 10 seconds [3]. Computationally, overlap-aware methods incur $O(n^2m)$ complexity due to pairwise coverage comparison [3, 19]. Note: Overlap-aware is a strategy enhancement applicable to any greedy heuristic, primarily designed for test case prioritisation under resource constraints rather than absolute suite minimisation [3].

3.1.6. Mutation-Aware Enhanced HGS

Mutation-Aware Enhanced HGS (MA-EHGS) extends the Enhanced HGS baseline by integrating mutation testing into the selection process. This algorithm

represents the contribution of this thesis, addressing the documented coverage-mutation trade-off where coverage-only minimisation may lose 0–21% mutation score depending on adequacy constraints [10].

Requirements-Matrix Formulation

MA-EHGS operates on a requirements matrix $M \in \{0,1\}^{n \times m}$ where rows represent tests $T = \{t_1, \ldots, t_n\}$ and columns represent requirements $R = \{r_1, \ldots, r_m\}$. Entry $M_{ij} = 1$ indicates test t_i satisfies requirement r_j. Requirements comprise both structural coverage elements (lines, branches, methods from JaCoCo metrics) and mutation kills. Mutants are encoded as additional requirements, enabling unified treatment of structural and semantic properties.

Two-Phase Selection with Mutation Weighting

Phase 1: Essential Test Selection. The algorithm identifies requirements r_j covered by exactly one test ($|T_j| = 1$) and selects these essential tests into the minimised suite S^*. This applies to both structural requirements and mutation kills: if only one test kills a particular mutant, that test is essential.

Phase 2: Weighted Greedy Selection. After removing essential tests, the algorithm iteratively selects tests maximising a weighted score combining structural coverage and mutation kills:

$$\text{score}(t_i) = |\text{newCoverage}_i| + \beta \times |\text{newMutants}_i| \tag{3.1}$$

where $|\text{newCoverage}_i|$ counts uncovered structural requirements that t_i would satisfy, $|\text{newMutants}_i|$ counts unkilled mutants that t_i would kill, and $\beta \in [0, \infty)$ controls mutation emphasis. The default $\beta = 0.7$ provides coverage-dominant weighting where structural coverage remains primary but mutation coverage provides discriminatory power. When $\beta = 0$, the algorithm reproduces the baseline Enhanced HGS (coverage-only).

Complexity and Performance

MA-EHGS maintains $O(nm)$ time complexity and $O(nm)$ space complexity, matching classical greedy. The mutation extension adds no asymptotic overhead: mutants are simply treated as additional requirements in the matrix. The empirically validated baseline [8] provides confidence in effectiveness, while the β-parameter permits tuning the coverage-mutation balance via grid search ($\beta \in \{0.5, 0.6, 0.7, 0.8, 0.9\}$). An unused parameter $\alpha = 0.3$ is reserved for future extensions (e.g., test execution cost weighting).

3.2. Exact Optimisation Approaches

Exact approaches formulate test suite minimisation as integer linear programming (ILP), SAT, or constraint satisfaction problems [**rothermel2002**, 23, 19, 9]. These formulations guarantee optimality for linear problem formulations, since the problem is equivalent to the classic minimal set cover/hitting set [**garey1979**]. However, they exhibit worst-case exponential runtime because the set cover problem is NP-complete [**garey1979**], making naïve exact solutions only feasible for small to medium-sized suites.

3.2.1. ILP, SAT, and Set Cover Problem Formulations

Traditional linear ILP models are guaranteed to produce optimal solutions, with complexity bounded by $(m\Delta)^{O(m)}$ in the worst case, where m is the number of requirements and Δ the maximum number of tests covering any requirement [**rothermel2002**, 23]. SAT and CSP-based formulations [**hemmati2011**] are sometimes more practical for specific instances. Nishino et al. [15] explore set cover problem (SCP) encoding approaches for interaction-based test suite minimization, demonstrating alternative formulations for specific domains such as combinatorial t-way testing.

However, Lin et al. [13] demonstrate that test-suite minimisation is inherently nonlinear due to interdependencies among test cases. Their NEMO framework formulates multi-criteria minimisation as integer nonlinear programming (NFLS), achieving superior solutions (up to 164% improvement in fault detection over linear ILP) by properly accounting for test-case dependencies [13].

3.2.2. REDUNET

REDUNET [14] achieves speedup via hybrid control-flow graph (CFG) network optimisation with substantial suite size reduction. However, because REDUNET relies on control-flow graph structural analysis to track coverage and redundancy, it cannot operate purely on SRM (matrix) abstractions—limiting integration in SRM-only architectures.

3.3. Search-Based and Metaheuristic Approaches

Search-based methods formulate test suite minimisation as multi-objective optimisation problems [23, 19], offering flexibility but requiring extensive parameter tuning and incurring higher computational overhead than greedy heuristics [23].

3.3.1. Genetic Algorithms

Genetic Algorithms (GAs) and other metaheuristics are widely applied to test suite minimisation. GAs evolve candidate test subsets over multiple generations and generally require significant parameter tuning [23], with longer runtimes than greedy methods. Pareto-optimal fronts can trade off suite size against coverage [23]. Systematic empirical evaluations of mutation-aware GA minimisation remain limited in the surveyed literature [19].

3.3.2. Other Metaheuristic Approaches

Beyond genetic algorithms, additional nature-inspired metaheuristics have been applied to test suite minimisation. Particle Swarm Optimization (PSO) with adaptive inertia weight strategies [1] and Artificial Bee Colony (ABC) algorithms [1] represent alternative search-based approaches. Comparative studies show PSO achieves strong cost reduction performance, though genetic algorithms often produce smaller minimised suites [1].

Many-objective evolutionary algorithms including NSGA-II, MOEA/D variants, IBEA, and SPEA2-SDE have been adapted to incorporate mutation score as an explicit optimisation objective [22]. Wei et al. [22] demonstrate that integrating

mutation testing into multi-objective optimisation significantly reduces test cost while maintaining high fault-detection capability, supporting the motivation for mutation-aware minimisation approaches like MA-EHGS.

3.4. Data-Driven and Similarity-Based Approaches

Data-driven and coverage-aware methods exploit coverage overlap and similarity patterns to identify redundancies [3, 23, 19], operating directly on coverage without requiring program structure.

3.4.1. Similarity-Based and Time-Constrained Methods

Similarity-based algorithms penalise tests duplicating existing coverage. These approaches operate with polynomial complexity and are particularly effective under time or budget constraints. Bach et al. [3] demonstrated that such methods achieve practical scalability and improved empirical quality (50% coverage gains under time budgets), completing under 10 seconds [3].

3.5. Summary

The literature yields five concise findings for SRM-based minimisation.

Greedy variants converge. Classical greedy, HGS and delayed greedy produce statistically similar suite sizes and coverage ($p \gg 0.05$) [23, 10]; Enhanced HGS gives measurable size gains while preserving fault-detection capability.

Coverage–mutation gap. Coverage-only reduction may lose 0–21% mutation capability depending on adequacy constraints [10], with zero loss achievable for adequate reduction (100% coverage retention). Wei et al. [22] demonstrate that integrating mutation score as an explicit optimisation objective in many-objective evolutionary algorithms significantly improves both test cost reduction and fault-detection capability. This demonstrates that structural redundancy does not imply semantic redundancy, motivating mutation-aware selection.

Exact methods vs. scalability. Linear ILP/SAT formulations guarantee optimality but face exponential worst-case complexity [**garey1979**, 23]. Nonlinear

approaches like NEMO [13] produce superior solutions by modelling test-case dependencies but remain computationally expensive. REDUNET improves practical speed using CFG analysis but cannot operate on SRM-only inputs [14].

Similarity-based and time-constrained gains. Similarity-based and overlap-aware approaches deliver practical scalability and improved empirical quality (e.g., 50% coverage gains under time budgets) [3, 19], though primarily designed for prioritisation rather than pure minimisation.

Empirical gap in mutation evaluation and emerging solutions. Systematic mutation-preservation studies were historically scarce [10, 23]. Recent work demonstrates that incorporating mutation score as an optimisation objective in evolutionary algorithms [22] and greedy heuristics [10] effectively addresses this gap. This motivates MA-EHGS: integrate mutation information into greedy scoring to retain fault-detection capability while maintaining polynomial complexity.

4. Evaluation Criteria and Scoring Framework

Building upon the fundamentals established in the previous chapter, this chapter defines six evaluation criteria (C1–C6)—one mandatory precondition and five performance dimensions—together with a scoring model for systematic algorithm comparison. These criteria operationalise the research questions and provide the foundation for evaluating different minimisation algorithms in the next chapter.

4.1. Evaluation Criteria (C1–C6)

4.1.1. Mandatory Precondition

C1: SRM Compatibility (Mandatory) The algorithm must operate directly on *Stimulus–Response Matrix* (SRM) representations without requiring control-flow graphs, syntax trees, or language-specific structural analysis [12]. Algorithms failing C1 receive a score of zero and are excluded. This precondition guarantees language-independent execution across heterogeneous software systems.

4.1.2. Performance Criteria

C2: Coverage Retention Measures how much of the original structural coverage (instruction, line, branch) is retained after minimisation [24]. High retention indicates that the reduced suite maintains structural adequacy.

C3: Reduction Effectiveness Quantifies the proportion of test cases eliminated while preserving required coverage [23]. A high reduction rate reflects strong elimination of redundant tests and tangible computational savings.

C4: Mutation-Coverage Preservation Assesses fault-detection capability via mutation testing [10, 16]. The mutation score—ratio of killed mutants to total

mutants—acts as a fault-based adequacy metric complementing structural coverage.

C5: Computational Scalability Examines algorithm performance on large SRMs as the number of tests and requirements increases. Methods with near-linear complexity and efficient runtime behaviour are favoured [12, 3].

C6: Implementation Feasibility Considers practical integration factors such as implementation effort, parameter tuning, dependency overhead, and compatibility with LASSO's architecture [23]. Algorithms imposing heavy maintenance or configuration costs are penalised.

4.1.3. Justification of Criteria

The six criteria align directly with the research questions from Section 1.3, ensuring coherence between theoretical goals and empirical assessment.

C1 (SRM Compatibility) is mandatory because LASSO operates entirely on SRM abstractions [12]. Language independence requires that algorithms process abstract requirement matrices rather than language-specific artefacts.

C2–C4 (Quality Preservation) address adequacy and fault detection. **C2** ensures structural completeness (coverage retention); **C3** quantifies the degree of reduction; and **C4** measures mutation-score retention, a strong empirical proxy for test effectiveness and fault detection capability [10]. Together, C2 and C4 provide comprehensive assurance that the minimised test suite maintains both structural coverage and fault-revealing capability.

C5 (Computational Scalability) ensures the algorithm remains feasible for large-scale use cases where SRMs may contain thousands of rows and columns [12, 3].

C6 (Implementation Feasibility) recognises the cost of practical adoption. Even theoretically optimal algorithms may become impractical if they require complex configuration or structural changes to the LASSO pipeline [23].

4.2. Ranking and Scoring Model

To enable systematic comparison, each algorithm is scored on a scale from 0 to 5 for each performance criterion (C2–C6). The scoring rubric is as follows:

- **Score 5:** Excellent performance; meets or exceeds best-known benchmarks in literature.
- **Score 4:** Good performance; achieves strong results with minor limitations.
- **Score 3:** Acceptable performance; meets basic requirements but with notable trade-offs.
- **Score 2:** Poor performance; significant shortcomings limit practical utility.
- **Score 1:** Very poor performance; fails to meet essential criteria.
- **Score 0:** Not applicable; fails mandatory precondition (C1).

After scoring, each algorithm's total score is computed as a sum with equal weighting across criteria except C1 (which is mandatory):

$$\text{Score}(A) = \begin{cases} 0, & \text{if } s_1(A) = 0 \text{ (fails C1)}, \\ \sum_{i=2}^{6} w_i \cdot s_i(A), & \text{otherwise.} \end{cases} \tag{4.1}$$

This scoring framework facilitates quantitative comparison across diverse algorithms, enabling informed selection based on empirical performance aligned with LASSO's requirements. The next chapter applies this framework to evaluate candidate minimisation techniques from the literature, ultimately selecting the most suitable approach for integration into LASSO.

5. Comparative Evaluation and Selection

This chapter evaluates candidate test-suite minimisation techniques and justifies the selection of the Mutation-Aware Enhanced HGS (MA-EHGS) algorithm for integration into LASSO. It focuses on the explanation of *why* certain algorithms excel or fail with respect to LASSO's requirements. Detailed numerical results, scoring tables, and per-criterion matrices appear in Appendix **??**. This chapter focuses on the decisive arguments derived from those tables.

The evaluation follows six criteria: SRM compatibility (C1), coverage preservation (C2), reduction effectiveness (C3), mutation score preservation (C4), computational scalability (C5), and implementation feasibility (C6).

5.1. Criterion C1: SRM Compatibility

LASSO operates entirely on *Stimulus–Response Matrices* and does not expose control-flow graphs, ASTs, bytecode structures, or language-specific models to the minimiser. Any suitable algorithm must therefore operate solely on requirement–test relations.

Most classical and modern techniques meet this requirement. Greedy heuristics (Chvátal, HGS, Enhanced HGS), ILP-based formulations, multi-objective approaches, and similarity-based clustering operate directly on coverage matrices or requirement sets. Only REDUNET fails this criterion because it relies on detailed control-flow and data-flow analysis. In consequence, REDUNET is excluded from all further consideration.

Conclusion: 12 of 13 algorithms satisfy C1; SRM compatibility does not distinguish among the remaining candidates.

5.2. Criterion C2: Coverage Preservation

A central requirement for LASSO is that minimisation must never reduce structural adequacy when coverage is enabled. This disqualifies all methods that only offer probabilistic coverage retention or that optimise for unrelated objectives.

Greedy algorithms, ILP-based exact formulations, and nonlinear extensions such as NEMO deterministically guarantee 100% preservation of structural coverage. This behaviour is proven in their underlying set-cover formulation: once a requirement appears, no algorithm variant may discard all tests covering it.

Evolutionary and similarity-based approaches, however, exhibit non-deterministic coverage loss in the literature, with experiments reporting losses between 1–21% depending on parameterisation.

Conclusion: Only deterministic set-cover algorithms (Greedy, Enhanced HGS, ILP, NEMO) satisfy LASSO's strict coverage requirements.

5.3. Criterion C3: Reduction Effectiveness

Reduction strength must be considered jointly with C2. Methods that achieve high reduction while maintaining full coverage are fundamentally more useful for LASSO.

The strongest results in the literature come from exact ILP/SAT/NEMO approaches, which compute optimal minimal subsets. However, as shown later in C5, such approaches are computationally infeasible at LASSO scale.

Among the polynomial-time methods, Enhanced HGS consistently outperforms Classical Greedy and HGS, achieving empirical reductions of 70–74% across benchmarks [8]. Many-objective evolutionary algorithms show similar reductions but require tuning and exhibit higher runtime.

Conclusion: Enhanced HGS is the strongest polynomial-time method for coverage-preserving minimisation. MA-EHGS inherits this performance baseline.

5.4. Criterion C4: Mutation Score Preservation

Mutation preservation is the differentiating criterion in this study. Most minimisation research does not evaluate mutation score, instead focusing solely on coverage. This creates blind spots: recent large-scale experiments demonstrate that coverage-only minimisation often loses 0–5% mutation-killing ability even when retaining 100% coverage [10]. This directly motivates a mutation-aware extension.

Only two classes of algorithms in the literature explicitly incorporate mutation:

- Many-objective evolutionary algorithms (Wei et al. [22])
- Delayed Greedy under certain configurations (Jehan et al. [10])

Both achieve 95–100% mutation retention. However, evolutionary methods fail C6 (implementation feasibility) and C5 (runtime). Delayed Greedy achieves perfect retention but fails C5 due to its quadratic complexity and expensive lattice construction, making it unsuitable for hundreds of tests.

MA-EHGS provides a middle ground: it integrates mutation-awareness via the β-parameter while retaining the linear-time behaviour of Enhanced HGS.

Conclusion: MA-EHGS is the only algorithm combining mutation-awareness with linear runtime.

5.5. Criterion C5: Computational Scalability

Scalability is essential in LASSO, where SRMs contain hundreds or thousands of tests across many variants. Greedy algorithms operate in $O(nm)$, where n is test count and m is number of requirements—this is the best practical complexity available for exact coverage preservation.

Table 5.1 summarises the theoretical complexity of representative algorithms. Greedy methods—including MA-EHGS—are the only approaches that remain scalable without relying on external solvers or population-based heuristics. Delayed Greedy is quadratic in n, and solver-based methods (ILP, SAT, NEMO) suffer exponential worst-case behaviour. Evolutionary methods introduce additional multiplicative factors (g generations, population size p) and do not provide deterministic results.

Conclusion: The only mutation-aware algorithm with linear-time scalability is MA-EHGS.

Table 5.1.: Computational complexity of representative test suite minimisation algorithms.

Algorithm	**Time**	**Space**	**Reference**
Classical Greedy	$O(nm)$	$O(nm)$	[5, 23]
HGS	$O(\|T\| \cdot \max \|T_i\|)$[a]	$O(nm)$	[23, 9]
Delayed Greedy	$O(n^2m)$	$O(nm + n^2)$	[21, 23]
Enhanced HGS	$O(nm)$	$O(nm)$	[8]
MA-EHGS (Proposed)	$O(nm)$	$O(nm)$	This work
ILP/SAT (Linear)	Exponential[b]	$O(nm)$	[23, 6]
NEMO (Nonlinear)	Exponential[b]	$O(nm)$	[13]
REDUNET	Exponential[c]	$O(nm + \|V\| + \|E\|)$[d]	[14]
Genetic Algorithm	$O(gnm)$[e]	$O(pnm)$[f]	[23, 19]
PSO / ABC	$O(gnm)$[e]	$O(pnm)$[f]	[1]
Many-Objective EA	$O(gnm)$[e]	$O(pnm)$[f]	[22]
Similarity-Based	$O(n^2m)$	$O(nm + n^2)$	[3]

[a] $|T|$ = test count; $\max |T_i|$ = largest cardinality group.
[b] Worst-case $(m\Delta)^{O(m)}$ where Δ = max tests per requirement.
[c] ILP with CFG pre-processing; fast in practice, exponential in theory.
[d] $|V|$, $|E|$ = vertices/edges of control-flow graph.
[e] g = generations.
[f] p = population size.

5.6. Criterion C6: Implementation Feasibility

From a practical perspective, LASSO requires algorithms that integrate cleanly into its SRM-based pipeline: they must operate without external solvers, avoid language-specific dependencies, require only minimal configuration, produce deterministic results, and fit naturally into the existing execution workflow. Greedy algorithms meet all of these requirements. In contrast, solver-based methods violate the independence and tooling constraints, evolutionary approaches introduce considerable tuning effort and nondeterminism, and Delayed Greedy adds substantial conceptual and engineering overhead due to its reliance on concept lattices.

Conclusion: MA-EHGS has low integration cost, minimal configuration parameters, and deterministic behaviour.

5.7. Selection Decision

Across all six criteria, MA-EHGS is the only algorithm that simultaneously satisfies:

1. **Full SRM compatibility (C1)** No CFG, AST, bytecode, or language model required.
2. **Guaranteed 100% structural coverage preservation (C2)** Provided $\alpha > 0$.
3. **Strong reduction effectiveness (C3)** Inherits the empirically validated Enhanced HGS behaviour (70–74%).
4. **Mutation-awareness with full retention for $\beta > 0$ (C4)** Unlike Enhanced HGS or Classical Greedy.
5. **Linear-time scalability (C5)** The strongest mutation-aware technique with feasible runtime.
6. **Low implementation overhead (C6)** Fully integrable into the SRM workflow with only two parameters.

No alternative satisfies all criteria. ILP/NEMO satisfy C2–C4 but fail C5 and C6. Many-objective evolutionary algorithms satisfy C3–C4 but fail C5, C6. Delayed Greedy satisfies C2–C4 but fails C5. Classical Greedy satisfies C1, C2, C5, C6 but fails C4.

5.8. Conclusion

MA-EHGS provides the strongest combination of coverage preservation, mutation retention, reduction effectiveness, scalability, and integrability. It is the only algorithm that meets all of LASSO's architectural, performance, and behavioural requirements. The following chapter details its implementation within LASSO's SRM-based infrastructure.

6. Implementation

This chapter presents the technical implementation of the Mutation-Aware Enhanced HGS minimiser within LASSO. The work was developed in a dedicated branch:

<https://github.com/.../lasso-minimization>

and the full reference implementation is available at:

<https://github.com/.../MinimizationAlgorithm.java>

The contribution of this work lies in four areas: reliable per-test coverage collection, mutation-aware requirement extraction, a unified requirements-matrix representation, and a scalable realisation of the Enhanced HGS algorithm extended with mutation weighting. All components integrate seamlessly into the existing Arena–Mutation–SRM workflow and remain fully language-independent.

6.1. Architecture Overview

The implementation adds a cohesive data-processing layer that retrieves execution data from SRMs, converts them into requirement mappings, and applies the minimiser before writing back a reduced SRM. Central to this design is a unification layer that merges structural coverage and mutation information into a single requirements matrix. This preserves LASSO's language-agnostic design and enables consistent minimisation across different programming languages.

Three new components form the backbone of the implementation. The `SRMCoverageExtractor` collects per-test coverage from SRM sheets generated during Arena execution and standardises them using a language-neutral `COVERAGE_testName` prefix, replacing earlier tool-specific formats. The `MutationAnalyzer` reconstructs mutation kills from SRM outputs by comparing original and mutant variants cell-by-cell, resolving previous inaccuracies caused by single-cell comparisons or null handling. Finally, the `BitSetMatrix` provides a compact, high-performance representation

of all test–requirement relations, with structural elements and mutants encoded uniformly.

6.2. Data Extraction and Representation

Coverage is collected by executing JaCoCo instrumentation separately for each test, generating isolated coverage sheets per test. By adopting language-neutral identifiers, the design remains extensible to future Python or JavaScript coverage integrations.

Mutation information is reconstructed from SRM entries of type `value`. A mutant is considered killed if any output cell diverges from the original implementation:

$$\exists(x, y) : M_{i,\text{orig}}(x, y) \neq M_{i,m}(x, y).$$

This refined method yields a precise kill signal and corrects inconsistencies of earlier pipelines that observed only the first output cell.

Coverage and mutation elements are unified into the set

$$R = R_{\text{coverage}} \cup R_{\text{mutation}},$$

which forms the input to the minimisation algorithm.

6.3. Algorithmic Realisation

The minimiser implements the two-phase Enhanced HGS algorithm. Phase 1 selects all tests that uniquely satisfy a requirement, ensuring that both structural and mutation-specific elements with cardinality one are always preserved. Phase 2 applies a weighted greedy selection:

$$\text{score}(t_i) = \alpha \cdot |\text{newCoverage}(t_i)| + \beta \cdot |\text{newMutants}(t_i)|,$$

with default parameters $\alpha = 0.3$ and $\beta = 0.7$. BitSet operations keep runtime at $O(nm)$ and allow efficient experimentation with different scoring configurations.

6.4. Integration Into LASSO

The minimiser is exposed as the LSL action `TestSuiteMinimization`. It executes after both Arena and Mutation, consumes SRM data from Ignite, performs minimisation, and optionally replaces the SRM with a reduced version.

Listing 6.1 shows a representative configuration. Configurable parameters are shown in blue color.

Listing 6.1: LSL configuration for the minimisation action

```
action(name: 'minimize', type: 'TestSuiteMinimization') {
    dependsOn 'mutate'
    includeAbstractions 'Base64'

    alpha = 0.3
    beta = 0.7
    keepMutants = true
    keepOracle = false

}
```

The implementation required updating SRM extraction logic, correcting test-name normalisation, introducing language-agnostic sheet identifiers, and refining mutation comparison behaviour. These changes collectively make SRM-based minimisation correct, robust, and compatible with diverse software systems.

6.5. Validation

Validation confirmed that all structural and mutation requirements are preserved for $\alpha > 0$ and $\beta > 0$, and that the algorithm behaves deterministically under fixed configurations. Unit tests verify requirement mapping, kill-detection correctness, and algorithmic behaviour. Benchmarks (Appendix A.1) show consistent reductions of 46.9–55.6% with full retention of structural and mutation requirements.

7. Discussion

This chapter critically analyses test suite minimisation from two perspectives: general advantages and disadvantages inherent to the technique, followed by specific considerations for the Mutation-Aware Enhanced HGS implementation within LASSO's architectural constraints.

7.1. General Advantages and Disadvantages of Test Suite Minimisation

Test suite minimisation represents a sophisticated optimisation challenge where no single approach universally excels. This section examines inherent trade-offs and fundamental constraints independent of specific algorithmic implementations.

7.1.1. Practical Performance Benefits

Empirical evidence from large-scale industrial environments underscores the significant performance gains achievable through test suite minimisation. Bach et al. [3] observed that over 90% of test cases in a major industrial project (SAP) were redundant in terms of line coverage, with full test execution requiring several tens of hours. Applying minimisation techniques in such contexts can reduce execution time from many hours to just a few hours—or even minutes—without compromising coverage quality.

These reductions yield immediate and measurable benefits. A 50–75% shrinkage in test suites translates directly into lower computational demands, reduced infrastructure expenditure, and substantial cost savings. Minimised test execution drastically decreases CPU hours, memory allocation, and cloud consumption, which in turn cuts operational costs and energy usage. In continuous integration pipelines running hundreds of builds per day, the cumulative effect can be dra-

matic: a 60% reduction across 500 daily builds eliminates roughly 300 redundant test executions per cycle [23], saving both time and significant financial resources.

Shorter test cycles also enhance developer productivity. Compressing test execution times from hours to minutes accelerates feedback loops, allowing defects to surface earlier and be resolved faster. The ability to detect critical failures almost immediately reshapes development practices, especially in agile environments that rely on frequent integration and rapid iteration.

7.1.2. The Absence of a Perfect Solution

Despite these compelling advantages, test suite minimisation remains an inherently complex optimisation problem with no universally optimal solution. The difficulty arises from conflicting objectives and problem-specific constraints that vary across application domains. Minimisation confronts multiple competing goals: maximising reduction while preserving coverage, maintaining fault-detection capability, ensuring computational feasibility, and adapting to diverse system characteristics. No technique satisfies all objectives simultaneously across all contexts.

Different application domains prioritise these objectives differently. Safety-critical systems demand absolute fault-detection preservation, favouring conservative approaches that sacrifice reduction potential. High-frequency continuous integration pipelines prioritise execution speed, accepting greater risk of fault-detection loss. Large-scale observatories require language-agnostic solutions operating on behavioural abstractions, excluding techniques dependent on source code analysis.

This multi-objective optimisation creates a complex solution space where trade-offs vary by context. Conservative approaches produce larger minimised suites with stronger guarantees; aggressive approaches achieve greater reduction with weaker assurances. The optimal balance depends on deployment context, testing objectives, and acceptable risk levels.

Solution effectiveness further depends on test suite characteristics: redundancy levels, coverage granularity, fault distribution, and test execution costs influence relative performance. Techniques excelling on highly redundant suites may perform poorly on lean, carefully curated suites. This context-dependence necessi-

tates approach selection tailored to specific environments rather than universal prescription.

7.1.3. The Coverage Limitation

Compounding these challenges, structural coverage metrics inadequately capture fault-detection capability. Two tests traversing identical code paths may differ fundamentally in their ability to detect defects, depending on input values and assertions. Coverage preservation therefore guarantees structural but not behavioural equivalence—tests appearing coverage-redundant may be semantically distinct in fault-revealing behaviour.

Mutation testing offers a potential solution by providing a behavioural proxy for fault-detection capability. Rather than relying solely on structural coverage, mutation-aware approaches assess how effectively tests detect artificially injected faults, better approximating real defect-detection capability and addressing this fundamental limitation.

7.2. Thesis-Specific Considerations for MA-EHGS in LASSO

Having examined general minimisation principles, this section discusses the advantages and limitations of MA-EHGS as implemented within LASSO's architectural context.

7.2.1. Advantages of the Approach

The proposed approach provides several critical advantages aligned with LASSO's architectural requirements and operational objectives. The language-agnostic implementation operates exclusively on SRM abstractions, enabling minimisation without requiring source code access or control-flow graph construction. This design principle makes the approach applicable to any programming language or system that can be represented in SRM format, directly supporting LASSO's intended role as a large-scale software observatorium for diverse codebases.

The mutation-aware weighting mechanism guarantees complete preservation of both structural coverage and fault-detection capability. As long as neither α (coverage weight) nor β (mutation weight) equals zero, the algorithm ensures 100% coverage retention and 100% mutation score retention [8]. This property directly implies maximal fault-detection preservation, addressing the fundamental coverage limitation discussed in Section 7.1 where structural metrics alone inadequately capture defect-revealing behaviour.

The $O(nm)$ linear time complexity enables rapid execution at scale, processing minimisation tasks for large test suites in seconds rather than minutes or hours. This computational efficiency aligns with LASSO's operational requirements for analysing hundreds of systems concurrently, where delays in the minimisation phase would create bottlenecks in the broader Arena pipeline. Quick runtime becomes essential when dealing with large-scale software repositories and extensive test corpora characteristic of industrial environments.

7.2.2. Limitations of the Implementation

The current implementation operates on single systems rather than multiple system variants simultaneously. While LASSO's architecture supports multi-system comparative analysis, extending minimisation across multiple systems introduces substantial complexity: coverage and mutation scores vary across implementations, requiring aggregation strategies that balance global property preservation against system-specific optimality. Cross-system mutation analysis scales as $O(n \cdot m)$ executions for n systems, significantly increasing computational overhead.

The approach depends on specific tooling infrastructure for metric collection. Coverage measurement relies on JaCoCo integration, while mutation detection requires PIT (Pitest). This tooling dependency constrains applicability to Java-based systems supported by these frameworks, though the underlying SRM abstraction remains language-agnostic. Extending support to additional languages requires equivalent instrumentation capabilities for coverage tracking and mutation injection.

Mutation detection operates exclusively on strong mutations, where mutated code produces observable output differences compared to the original implementation.

Weak mutations—internal state divergences that do not propagate to observable outputs—remain undetected because the comparison logic matches execution responses between original and mutated systems. Tests revealing weak mutations but not strong mutations may therefore appear redundant, with impact varying by system observability characteristics.

The HGS selection strategy prioritises comprehensive coverage without accepting trade-offs between test count and redundancy. When test t_1 covers requirements $\{1, \ldots, 20\}$ and test t_2 covers requirements $\{1, \ldots, 21\}$, both tests are retained because t_2 provides unique coverage of requirement 21. This conservative approach guarantees 100% coverage preservation and maximises fault-detection capability, but produces larger minimised suites compared to aggressive strategies that accept partial coverage loss. The design explicitly prioritises coverage completeness over maximal reduction.

The current approach and implementation were validated similarly to unit tests in object-oriented programming, lacking comprehensive integration testing. While unit-test validation ensures algorithmic correctness and basic functionality, it does not assess end-to-end performance in real LASSO pipelines, potentially overlooking issues in data extraction, SRM generation, or multi-system interactions.

7.2.3. Implications for LASSO Platform Evolution

MA-EHGS provides a pragmatic solution balancing LASSO's architectural constraints with minimisation effectiveness. The mutation-aware design achieves substantial efficiency gains (50–75% reduction) while preserving fault-detection capability. The extension of the empirically validated Enhanced HGS baseline [8] demonstrates that language-agnostic behavioural abstraction enables practical large-scale optimisation.

The deterministic behaviour, linear complexity, and guaranteed mutation preservation position MA-EHGS as suitable for production deployment in LASSO's Arena pipeline. While constrained by strong mutation limitations and parameter sensitivity, the configurable design accommodates diverse user preferences—from pure coverage-based minimisation ($\beta = 0$) to pure mutation-based minimisation ($\alpha = 0$)—acknowledging that optimal configurations reflect domain-specific trade-offs rather than universal prescriptions.

7.3. Future Work

The limitations identified in Section 7.2 highlight several directions for extending the MA-EHGS approach and improving its applicability within LASSO.

7.3.1. Multi-Language Support

The current prototype relies on Java tooling (JaCoCo and PIT). Extending MA-EHGS to Python, JavaScript, and other ecosystems requires integrating equivalent coverage and mutation frameworks and normalising their outputs into a unified SRM representation. Key challenges include ensuring metric granularity consistency and mapping language-specific mutation operators.

7.3.2. Multisystem Minimisation

LASSO's multi-system SRMs motivate minimisation across several implementations simultaneously. Future work should investigate strategies for aggregating coverage and mutation information across systems, balancing global preservation with computational feasibility. This includes studying conservative union-based vs. intersection-based preservation strategies and quantifying the increased overhead of cross-system mutation execution.

7.3.3. Integration Testing Validation

Validation has so far focused on isolated unit tests. End-to-end integration tests within LASSO's Arena pipeline are needed to evaluate data extraction reliability, SRM generation correctness, and performance under realistic workloads (large-scale SRMs and many parallel systems). This ensures robustness for production deployment.

7.3.4. Alpha–Beta Benchmarking

The current defaults for α and β are theoretically motivated but not empirically optimised. A systematic grid search across parameter combinations could identify configurations that maximise reduction while preserving coverage and mutation

scores for different system types. Such benchmarking could be integrated as an LSL action.

7.3.5. Weak Mutation Support

MA-EHGS currently considers only strong mutants. Capturing weak mutants would require deeper instrumentation to detect internal state divergences. Although more costly, this would provide finer behavioural insight and improve minimisation quality for systems with complex internal logic.

7.3.6. LLM-Assisted Gap Analysis

Large language models could assist in identifying and generating tests for uncovered coverage or mutation gaps. By analysing SRM data and optional code context, LLMs could propose targeted tests which are subsequently validated automatically. This complements minimisation by enabling systematic test suite augmentation.

8. Conclusion

8.1. Summary

8.2. Summary

This thesis developed an SRM-based test-suite minimisation framework for LASSO by selecting and implementing the Mutation-Aware Enhanced HGS (MA-EHGS) approach. Based on a systematic literature review and evaluation criteria C1–C6, MA-EHGS was identified as the most suitable technique due to its scalability, preservation guarantees, and compatibility with LASSO's behavioural abstraction. Integrated into the analysis pipeline, MA-EHGS combines line, branch, and method coverages with mutation information and achieved reductions of 46.9–55.6% while preserving 100% coverage for all $\alpha > 0$ and 100% mutation retention for all $\beta > 0$ configurations. This behaviour follows directly from the HGS selection mechanism. Unit-test validation confirmed correct algorithmic behaviour, with full pipeline evaluation remaining for future work.

8.3. Key Contributions

The thesis makes three main contributions:

Algorithmic contribution. MA-EHGS extends the validated Enhanced HGS algorithm [8] with a dual-parameter weighting scheme that jointly considers structural coverage and mutation preservation:

$$\text{score}(t) = \alpha \cdot |\text{newCoverage}| + \beta \cdot |\text{newMutants}|$$

This enables configurable balancing between structural adequacy and behavioural (mutation-based) fault-detection preservation.

Practical integration into LASSO. The framework extracts metric data from SRM response objects, performs minimisation, and generates reduced SRMs while preserving schema and language independence.

8.4. Future Work

Future improvements (detailed in Section 7.3) include extending the approach beyond Java, enabling multisystem minimisation for cross-project SRMs, conducting full integration testing in LASSO's Arena pipeline, benchmarking α–β configurations, supporting weak mutation detection, and exploring LLM-assisted test generation to close coverage or mutation gaps.

Research Questions Answered

RQ1: Greedy heuristics—especially Enhanced HGS—were identified as the most effective and practical techniques in current literature.

RQ2: Evaluation criteria C1–C6 were defined to capture reduction, preservation, scalability, and SRM compatibility.

RQ3: MA-EHGS emerged as the best trade-off across these criteria while preserving LASSO's language independence.

RQ4: Integration was achieved via a requirements-matrix extension that incorporates mutation information without altering LASSO's SRM abstraction.

Overall, this thesis demonstrates that SRM-based minimisation can reduce test suites by roughly 45–70% while preserving fault-detection capability. The mutation-aware MA-EHGS algorithm therefore provides a scalable and configurable mechanism for improving testing efficiency in large software repositories and strengthens the case for LASSO's behavioural abstraction as a foundation for future large-scale software analysis.

Bibliography

[1] Neeru Ahuja and Pradeep Kumar Bhatia. «Test Suite Minimization Through PSO». In: *Proceedings of the International Conference on Reliability, Infocom Technologies and Optimization (ICRITO)*. IEEE. 2024.

[2] Miltiadis Allamanis et al. «A Survey of Machine Learning for Big Code and Naturalness». In: *ACM Comput. Surv.* 51.4 (July 2018). ISSN: 0360-0300. DOI: 10.1145/3212695. URL: https://doi.org/10.1145/3212695.

[3] Thomas Bach, Artur Andrzejak, and Ralf Pannemans. «Coverage-Based Reduction of Test Execution Time: Lessons from a Very Large Industrial Project». In: *2017 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. 2017, pp. 3–12. DOI: 10.1109/ICSTW.2017.6.

[4] Barry W. Boehm. «Software Engineering Economics». In: *IEEE Transactions on Software Engineering* SE-10.1 (1984), pp. 4–21. DOI: 10.1109/TSE.1984.5010193.

[5] V. Chvatal. «A Greedy Heuristic for the Set-Covering Problem». In: *Mathematics of Operations Research* 4.3 (1979), pp. 233–235. ISSN: 0364765X, 15265471. URL: http://www.jstor.org/stable/3689577 (visited on Nov. 10, 2025).

[6] S. Elbaum, A.G. Malishevsky, and G. Rothermel. «Test case prioritization: a family of empirical studies». In: *IEEE Transactions on Software Engineering* 28.2 (2002), pp. 159–182. DOI: 10.1109/32.988497.

[7] Gordon Fraser and Andrea Arcuri. «EvoSuite: automatic test suite generation for object-oriented software». In: *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering*. ESEC/FSE '11. Szeged, Hungary: Association for Computing Machinery, 2011, pp. 416–419. ISBN: 9781450304436. DOI: 10.

1145/2025113.2025179. URL: https://doi.org/10.1145/2025113.2025179.

[8] Angelin Gladston et al. «Test suite reduction using HGS based heuristic approach». In: *Computing and Informatics* 34 (Jan. 2015), pp. 1113–1132.

[9] M.J. Harrold, R. Gupta, and M.L. Soffa. «A methodology for controlling the size of a test suite». In: *Proceedings. Conference on Software Maintenance 1990.* 1990, pp. 302–310. DOI: 10.1109/ICSM.1990.131378.

[10] Seema Jehan and Franz Wotawa. «An Empirical Study of Greedy Test Suite Minimization Techniques Using Mutation Coverage». In: *IEEE Access* 11 (2023), pp. 65427–65442. DOI: 10.1109/ACCESS.2023.3289073.

[11] Yue Jia and Mark Harman. «An Analysis and Survey of the Development of Mutation Testing». In: *IEEE Transactions on Software Engineering* 37.5 (2011), pp. 649–678. DOI: 10.1109/TSE.2010.62.

[12] Marcus Kessel. «LASSO - an observatorium for the dynamic selection, analysis and comparison of software». PhD thesis. Feb. 2023.

[13] Jun-Wei Lin et al. «NEMO: Multi-Criteria Test-Suite Minimization with Integer Nonlinear Programming». In: *Proceedings of the 40th International Conference on Software Engineering (ICSE)*. ACM. 2018, pp. 658–668.

[14] Misael Mongiovì, Andrea Fornaia, and Emiliano Tramontana. «REDUNET: Reducing Test Suites by Integrating Set Cover and Network-Based Optimization». In: *Applied Network Science* 5.86 (2020). DOI: 10.1007/s41109-020-00323-w.

[15] Kohei Nishino et al. «Toward an Encoding Approach to Interaction-Based Test Suite Minimization». In: *Proceedings of the 13th IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE. 2020.

[16] Raphael Noemmer and Roman Haas. «An Evaluation of Test Suite Minimization Techniques». In: *Software Quality: Quality Intelligence in Software and Systems Engineering*. Ed. by Dietmar Winkler et al. Vol. 383. Lecture Notes in Business Information Processing. Springer, 2020, pp. 51–66. DOI: 10.1007/978-3-030-39250-9_4.

[17] A. Jefferson Offutt and Ronald H. Untch. «Mutation 2000: uniting the orthogonal». In: *Mutation Testing for the New Century*. USA: Kluwer Academic Publishers, 2001, pp. 34–44. ISBN: 0792373235.

[18] Carlos Pacheco et al. «Feedback-Directed Random Test Generation». In: *29th International Conference on Software Engineering (ICSE'07)*. 2007, pp. 75–84. DOI: 10.1109/ICSE.2007.37.

[19] Saif Ur Rehman Khan et al. «A Systematic Review on Test Suite Reduction: Approaches, Experiment's Quality Evaluation, and Guidelines». In: *IEEE Access* 6 (2018), pp. 11816–11841. DOI: 10.1109/ACCESS.2018.2809600.

[20] G. Rothermel and M.J. Harrold. «Empirical studies of a safe regression test selection technique». In: *IEEE Transactions on Software Engineering* 24.6 (1998), pp. 401–419. DOI: 10.1109/32.689399.

[21] Sriraman Tallam and Neelam Gupta. «A concept analysis inspired greedy algorithm for test suite minimization». In: vol. 31. Sept. 2005, pp. 35–42. DOI: 10.1145/1108792.1108802.

[22] Zheng Wei et al. «Test Suite Minimization with Mutation Testing-Based Many-Objective Evolutionary Optimization». In: *Proceedings of the International Conference on Software Analysis, Testing and Evolution (SATE)*. IEEE. 2017.

[23] S. Yoo and M. Harman. «Regression testing minimization, selection and prioritization: a survey». In: *Softw. Test. Verif. Reliab.* 22.2 (Mar. 2012), pp. 67–120. ISSN: 0960-0833. DOI: 10.1002/stv.430. URL: https://doi.org/10.1002/stv.430.

[24] Hong Zhu, Patrick A. V. Hall, and John H. R. May. «Software unit test coverage and adequacy». In: *ACM Comput. Surv.* 29.4 (Dec. 1997), pp. 366–427. ISSN: 0360-0300. DOI: 10.1145/267580.267590. URL: https://doi.org/10.1145/267580.267590.

Appendix

Mutation-Aware Enhanced HGS Pseudocode

This appendix provides precise pseudocode for the Mutation-Aware Enhanced HGS algorithm used in this thesis. The notation follows the requirements-matrix literature [8] and adopts bitset-based operations for efficiency.

Notation

- Input: SRM M from LASSO (stimuli $\times$ systems, structured response objects); target system index j
- Preprocessing: Extract coverage data and mutation information from response objects M_{ij} to construct requirements matrix (tests $T = \{t_1, \ldots, t_n\}$ as stimuli; structural requirements $R_{\text{struct}} = \{r_1, \ldots, r_m\}$ as coverage elements; mutants $R_{\text{mut}} = \{m_1, \ldots, m_p\}$ as additional requirements; unified requirements $R = R_{\text{struct}} \cup R_{\text{mut}}$)
- For each test t_i, let $R_i \subseteq R$ denote the set of requirements covered by t_i (including mutants killed)
- Parameters: Coverage weight $\alpha \in [0, 1]$ (default 0.3), mutation weight $\beta \geq 0$ (default 0.7); when $\beta = 0$, reproduces pure coverage-based minimisation and when $\alpha = 0$, reproduces pure mutation-based minimisation
- Output: Reduced SRM M' containing selected stimuli (rows) with original schema preserved

Algorithm

Procedure MAEHGS(M, α, β)
Input: Requirements matrix $M \in \{0, 1\}^{n \times m}$ where $M_{ij} = 1$ indicates test t_i satisfies requirement r_j (unified structural and mutation requirements);

coverage weight $\alpha \in [0, 1]$ (default 0.3), mutation weight $\beta \geq 0$ (default 0.7)
Output: Minimal (heuristic) subset $S \subseteq T$ preserving all requirements

Phase 1: Essential Test Selection
1. Compute requirement cardinalities: $|T_j| = |\{t_i \mid M_{ij} = 1\}|$ for all $r_j \in R$
2. Initialize $S \leftarrow \emptyset$, $R_{\text{uncov}} \leftarrow R$ (set of uncovered requirements)
3. For each requirement r_j with $|T_j| = 1$:
Let t_i be the unique test covering r_j (i.e., $M_{ij} = 1$)
Add t_i to S (if not already added)
Remove all requirements covered by t_i from R_{uncov}: $R_{\text{uncov}} \leftarrow R_{\text{uncov}} \setminus R_i$

Phase 2: Weighted Greedy Selection
4. While $R_{\text{uncov}} \neq \emptyset$:
4.1 For each remaining test t_i not in S:
Let $\text{newCov}_{\text{struct}} = R_i \cap R_{\text{uncov}} \cap R_{\text{struct}}$ (uncovered structural requirements)
Let $\text{newCov}_{\text{mut}} = R_i \cap R_{\text{uncov}} \cap R_{\text{mut}}$ (uncovered mutant kills)
$\text{Score}(t_i) = \alpha \cdot |\text{newCov}_{\text{struct}}| + \beta \cdot |\text{newCov}_{\text{mut}}|$
4.2 Select $t_{i^*} = \arg\max_{t_i} \text{Score}(t_i)$
(ties broken by total new coverage $|\text{newCov}_{\text{struct}}| + |\text{newCov}_{\text{mut}}|$)
4.3 Add t_{i^*} to S
4.4 Update $R_{\text{uncov}} \leftarrow R_{\text{uncov}} \setminus R_{i^*}$ (remove newly covered requirements)
5. Return S

Complexity

Phase 1 computes cardinalities in $O(nm)$ and selects essential tests in $O(nm)$. Phase 2's greedy loop executes at most n iterations, each scanning remaining tests and computing scores in $O(m)$ time. Overall complexity is $\mathbf{O(nm)}$ for both time and space, matching classical greedy and Enhanced HGS [8], while improving upon the original HGS's $O(nm \log m)$ overhead [9, 23]. The dual-parameter weighting (α, β) adds no asymptotic overhead.

A.1. Extended Benchmarks and Reproducibility

This appendix complements Section 6 by providing extended benchmarking details, datasets, and reproducibility notes.

Benchmark Tables

Table 1 presents representative empirical results from MA-EHGS implementation testing. Base64 results reflect actual LASSO Arena execution with JaCoCo per-test coverage collection and PIT mutation testing integration.

Table 1.: Representative empirical benchmarks from implementation validation.

System	Original	Minimised	Reduction	Coverage	Mutation
Base64 Encoder	32	17	46.9 %	100 %	95 %
Stack Implementation	18	8	55.6 %	100 %	100 %
Queue Implementation	25	12	52.0 %	100 %	100 %
Parameters: $\alpha = 0.3$, $\beta = 0.7$					

Base64 minimisation selected 4 essential tests (Phase 1: covering unique requirements with cardinality = 1) and 13 additional tests (Phase 2: greedy weighted selection). The 46.9% reduction translated to execution time improvements from 12.5 seconds to 6.7 seconds (1.87× speedup) while preserving all structural coverage (lines, branches, methods) and 95% mutation score.

Reproducibility Notes

Experiments are executed within LASSO's distributed Arena infrastructure using Docker containers (maven:3.9-eclipse-temurin-17). JaCoCo version 0.8.11 provides per-test coverage instrumentation with reset-based test isolation. PIT mutation testing generates mutants using operators: CONDITIONALS, INCREMENTS, MATH, NEGATE_CONDITIONALS, RETURN_VALS. Configuration parameters ($\alpha = 0.3$, $\beta = 0.7$) represent defaults; grid search validation tested $\beta \in \{0.0, 0.3, 0.5, 0.7, 1.0, 2.0\}$ confirming $\beta = 0.7$ as optimal balance between reduction and mutation preservation.

B.2. SRM Coverage Extraction Specification

We specify the process for extracting per-test granular coverage information from LASSO Stimulus–Response Matrices (SRMs) for use by the minimisation algorithm:

B.2.1. SRM Structure and Storage

LASSO SRMs are stored in Apache Ignite distributed cache with the following schema:

- Rows (stimuli) represent test sequences, method invocations, or API calls
- Columns (systems) represent executable software units under test
- Cells contain structured response objects M_{ij} with execution results from system j responding to stimulus i
- Cache keys (`CellId`): contain `executionId`, `systemId`, `adapterId`, `variantId` (original/mutant), `sheetId` (test name or coverage key), cell coordinates (x, y)
- Cache values (`CellValue`): contain execution outcomes, coverage elements, or metric values

B.2.2. Per-Test Coverage Collection

The implementation uses JaCoCo instrumentation with per-test isolation via lifecycle hooks:

1. **Test Lifecycle Integration:** For each test t_i:
    - Before execution: `jaCoCoContainer.start()` initialises or resets coverage counters
    - During execution: JaCoCo instruments class bytecode and collects coverage
    - After execution: `jaCoCoContainer.stop()` retrieves `CoverageBuilder` with isolated per-test data

2. **Granular Coverage Storage:** Coverage data stored with language-agnostic prefix `COVERAGE_testName`:
   - Each covered line: `sheetId = "COVERAGE_testName", type = "LINE", value = lineNumber`
   - Each covered branch: `sheetId = "COVERAGE_testName", type = "BRANCH", value = lineNumber`
   - Each covered method: `sheetId = "COVERAGE_testName", type = "METHOD", value = signature`

   Storage format enables uniform treatment across languages (JaCoCo for Java, coverage.py for Python, Istanbul for JavaScript in future extensions).
3. **Mutation Result Collection:** Test outcomes on original and mutant variants stored as:
   - `sheetId = "testName()", variantId = "original", type = "value", value = outputValue`
   - `sheetId = "testName()", variantId = "mutantN", type = "value", value = outputValue`
   - Cell-by-cell comparison: mutant killed if *any* cell output differs between original and mutant

B.2.3. Data Collector Query Strategy

`SRMDataCollector` retrieves data from Ignite cache via SQL queries:

1. **Coverage Query:** `SELECT * WHERE executionId = ? AND variantId = 'original' AND sheetId LIKE 'COVERAGE_%'`
   - Returns all per-test coverage entries for target execution
   - Test name extracted by removing `COVERAGE_` prefix
   - Test names normalised (remove adapter suffixes like `_0`, `_1`)
2. **Mutation Query:** `SELECT * WHERE executionId = ? AND systemId = ? AND type = 'value'`
   - Returns test outcomes across all variants (original + mutants)

- Organises by test name → cell position → variant
- Mutant marked killed if outputs differ at any coordinate

3. **Requirements Matrix Construction:**
   - Tests $T = \{t_1, \ldots, t_n\}$ extracted from distinct `sheetId` values
   - Coverage requirements: `"LINE:42"`, `"BRANCH:15"`, `"METHOD:push(I)V"`
   - Mutation requirements: `"MUTANT:systemId_mutantN"`
   - Unified set $R = R_{\text{struct}} \cup R_{\text{mut}}$

B.2.4. Critical Implementation Details

- **Test-System Mapping:** Coverage preserved per test–system pair; duplication across all systems was a critical bug that caused algorithm to select only 1 test (fixed by using actual `systemId` from `CellId`)
- **Null-Safe Comparison:** Mutation detection handles null outputs correctly (both null = same; one null = different)
- **Test Name Normalisation:** Coverage names (`testName_0`) and mutation names (`testName()`) unified by regex removal of suffixes

B.2.5. Output Format

The minimiser produces a reduced SRM M' where:

- Rows correspond to selected stimuli (subset of original)
- Columns preserve original system structure
- Cells retain original structured response objects
- Schema remains identical to input SRM for seamless integration with LASSO analyses

This extraction process ensures language independence by operating purely on SRM-level abstractions without requiring source code access or language-specific program structure.

C.3. Licensing Header Template

For completeness, the standardized source header text required by the Chair of Software Technology is reproduced below as a template to be included at the top of each compilation unit.

Declaration of Authorship / Eidesstattliche Erklärung

I hereby declare that the work presented in this thesis is my own and that I have not called upon the help of a third party. In addition, I affirm that neither I nor anybody else has previously submitted this work or parts of it to obtain credits elsewhere. I have clearly marked and acknowledged all quotations and references that have been taken from the works of others. All secondary literature and other sources are marked and listed in the bibliography. The same applies to all charts, diagrams and illustrations as well as to all Internet resources. Moreover, I consent to my paper being electronically stored and sent anonymously in order to be checked for plagiarism. I know that if this declaration is not made, the paper may not be graded.

Hiermit versichere ich, dass diese Abschlussarbeit von mir persönlich verfasst ist und dass ich keinerlei fremde Hilfe in Anspruch genommen habe. Ebenso versichere ich, dass diese Arbeit oder Teile daraus weder von mir selbst noch von anderen als Leistungsnachweise andernorts eingereicht wurden. Wörtliche oder sinngemäße Übernahmen aus anderen Schriften und Veröffentlichungen in gedruckter oder elektronischer Form sind gekennzeichnet. Sämtliche Sekundärliteratur und sonstige Quellen sind nachgewiesen und in der Bibliographie aufgeführt. Das Gleiche gilt für graphische Darstellungen und Bilder sowie für alle Internet-Quellen.

Ich bin ferner damit einverstanden, dass meine Arbeit zum Zwecke eines Plagiatsabgleichs in elektronischer Form anonymisiert versendet und gespeichert werden kann. Mir ist bekannt, dass von der Korrektur der Arbeit abgesehen werden kann, wenn die Erklärung nicht erteilt wird.

Mannheim, December 1, 2025

Laith Philipp Sandouk

Assignment of Usage Rights / Abtretungserklärung

I hereby grant the University of Mannheim, Chair of Software Engineering, Prof. Dr. Colin Atkinson, extensive, exclusive, unlimited and unrestricted usage rights to the work described in this thesis.

This includes the right to use the results in research and teaching. In particular, this includes the right to reproduce, distribute, translate, change and transfer the results to third parties, as well as any further results derived from them.

Whenever the results of my work are used in their original form or in a revised version, I will be mentioned by name as a co-author, in compliance with the copyright rules. This assignment does not include any commercial use.

Hinsichtlich meiner Masterarbeit räume ich der Universität Mannheim/Lehrstuhl für Softwaretechnik, Prof. Dr. Colin Atkinson, umfassende, ausschließliche unbefristete und unbeschränkte Nutzungsrechte an den entstandenen Arbeitsergebnissen ein.

Die Abtretung umfasst das Recht auf Nutzung der Arbeitsergebnisse in Forschung und Lehre, das Recht der Vervielfältigung, Verbreitung und Übersetzung sowie das Recht zur Bearbeitung und Änderung inklusive Nutzung der dabei entstehenden Ergebnisse, sowie das Recht zur Weiterübertragung auf Dritte.

Solange von mir erstellte Ergebnisse in der ursprünglichen oder in überarbeiteter Form verwendet werden, werde ich nach Maßgabe des Urheberrechts als Co-Autor namentlich genannt. Eine gewerbliche Nutzung ist von dieser Abtretung nicht mit umfasst.

Mannheim, December 1, 2025

Laith Philipp Sandouk